

■■ Project 3: E-Commerce Relational Data Analysis (SQL)

■ Project Overview

Welcome to the implementation portfolio for **Project 3: SQL Data Analysis**, completing the data extraction phase of the DecodeLabs Industrial Training Track.

Building upon the pristine dataset finalized in Project 1 (Data Cleaning) and analyzed visually in Project 2 (EDA), this module shifts business operations away from flat, single-sheet Excel models into a scalable, structured relational database layer. Using an in-memory **SQLite3 database engine**, this project replicates corporate warehouse scenarios to execute high-performance transactional queries via raw structural text syntax.

■ Repository Blueprint & Architecture

To maintain enterprise-grade structure, organize your files within the repository using the following architecture:

```
■ e-commerce-sql-analysis-project3
■
■■■ ■ data/
■ ■■■ Cleaned_Dataset_Project1_Final.xlsx # Cleaned transaction master records
■
■■■ ■ queries/
■ ■■■ analytical_queries.sql # Standalone pure production SQL script
■
■■■ SQL_Project_3.ipynb # Live Jupyter Notebook with embedded SQLite3 engine
■■■ README.md # Comprehensive project report
```

■■ Relational Schema Data Definition Language (DDL)

To transition the flat spreadsheet structures into a queryable database environment, the data schema was mapped to a relational layout using the following SQL definitions:

```
CREATE TABLE transactions (
  OrderID VARCHAR(50) PRIMARY KEY,
  Date DATE,
  CustomerID VARCHAR(50),
  Product VARCHAR(100),
  Quantity INT,
  UnitPrice DECIMAL(10,2),
  ShippingAddress VARCHAR(255),
  PaymentMethod VARCHAR(100),
  OrderStatus VARCHAR(50),
  TrackingNumber VARCHAR(100),
  ItemsInCart INT,
  CouponCode VARCHAR(50),
  ReferralSource VARCHAR(100),
  TotalPrice DECIMAL(10,2)
);
```

■ Core Analytics Queries & Code Execution

Below are the production-grade SQL queries engineered to address the core business requirements set by the DecodeLabs training manual. These show filtering, sorting, and aggregate math across **1,200 unique records**.

1. Basic Data Selection & Validation

```
SELECT OrderID, Date, CustomerID, Product, TotalPrice
FROM transactions
LIMIT 5;
```

2. High-Yield Delivered Order Isolation (WHERE clause filtering)

```
SELECT OrderID, CustomerID, Product, Quantity, TotalPrice
FROM transactions
WHERE TotalPrice >= 2000.00 AND OrderStatus = 'Delivered'
ORDER BY TotalPrice DESC;
```

3. Sorting Financial Records (ORDER BY)

```
SELECT OrderID, Date, Product, TotalPrice
FROM transactions
ORDER BY TotalPrice DESC
LIMIT 10;
```

4. Categorical Metric Summarization (GROUP BY with Aggregations)

```
SELECT
    Product,
    COUNT(OrderID) AS Total_Orders_Placed,
    ROUND(AVG(UnitPrice), 2) AS Average_Unit_Cost,
    ROUND(SUM(TotalPrice), 2) AS Cumulative_Gross_Revenue
FROM transactions
GROUP BY Product
ORDER BY Cumulative_Gross_Revenue DESC;
```

■ Extracted Data Insights Table

Product Category	Total Orders Placed	Average Unit Cost	Cumulative Gross Revenue
Chair	178	\$366.52	\$195,620.11
Printer	181	\$377.92	\$195,612.61
Laptop	173	\$376.62	\$192,126.56
Tablet	179	\$352.02	\$186,568.95
Monitor	163	\$365.17	\$175,651.41
Desk	170	\$337.89	\$167,459.93
Phone	156	\$323.01	\$151,722.39

■ Top Strategic Business Observations

1. **Balanced Product Revenue Distribution:** Cumulative revenue is spread remarkably evenly across the top categories. Chairs (\$195,620.11) and Printers (\$195,612.61) lead aggregate sales by a razor-thin margin, showing strong portfolio diversification that protects the business from single-product supply chain drops.
2. **High-Value B2B Target Segment:** Using the WHERE threshold query, a specific cluster of repeating customer profiles emerged who consistently buy the maximum order cap (Quantity = 5) of high-tier items. This indicates a clear opportunity to establish a dedicated account management program or B2B sales pipeline.
3. **Optimized Checkout Size:** The relational average metrics show healthy cross-selling and bundling strategy execution, with an average ticket value of \$1,053.97 per transaction.

■ How to Run this Project Locally

1. Ensure you have **Python 3.x**, **Jupyter Notebook**, and **Pandas** installed.
2. Clone this repository to your local directory.
3. Launch the `SQL_Project_3.ipynb` notebook.
4. Run the initialization cells. The pipeline will automatically load `Cleaned_Dataset_Project1_Final.xlsx` and transform it into a live, active SQLite database table in your system's memory (RAM) so you can run queries instantly!